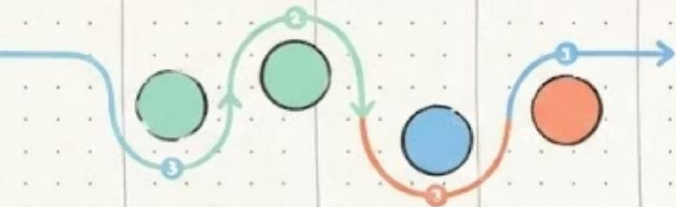


Gestão do Ciclo de Vida da Aplicação

Design Patterns (Parte 2)

Professor Romário da Silva Borges



Estruturais

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++. The code is mostly illegible due to its low opacity and the motion blur effect, but it adds a technical, digital feel to the overall composition.

Os padrões estruturais compreendem padrões que propõem soluções flexíveis para composição de classes e objetos. São eles: Proxy, Adapter, Facade, Decorator, Bridge, Composite e Flyweight.

3 - Proxy

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and depth. Scattered throughout the background are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to its low opacity and the overall aesthetic of the image.

Contexto: Função que pesquisa livros

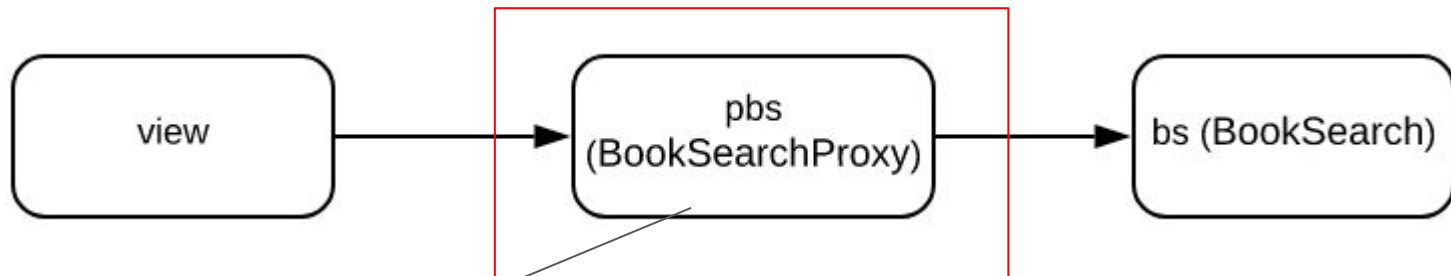
```
class BookSearch {  
    ...  
    Book getBook(String ISBN) { ... }  
    ...  
}
```

Problema: Inserir um cache (para melhorar desempenho)

- Se "livro no cache"
 - retorna livro imediatamente
 - caso contrário, continua com a pesquisa
- Porém, não queremos modificar o código de BookSearch
 - Classe já está funcionando
 - Já tem um desenvolvedor acostumado a mantê-la, etc

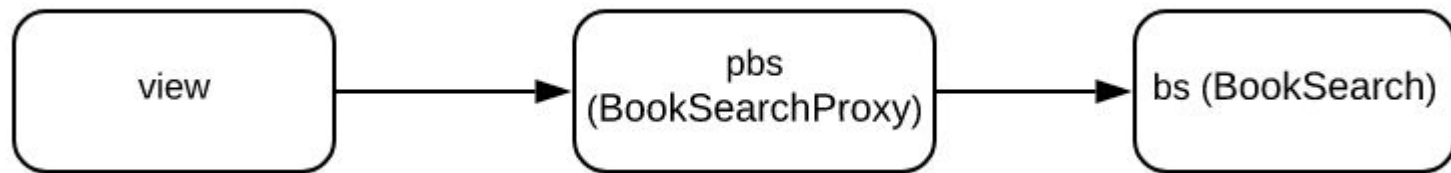
Solução: Padrão Proxy

- Proxy: objeto intermediário entre cliente e um objeto base
- Clientes não "falam" mais com o objeto base (diretamente)
- Eles têm que passar antes pelo proxy

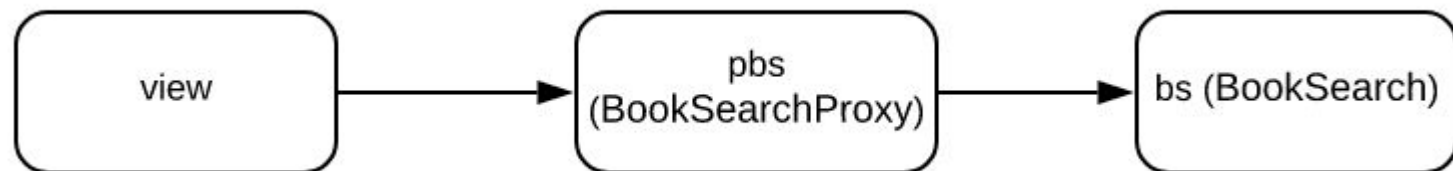


Implementa a lógica do cache

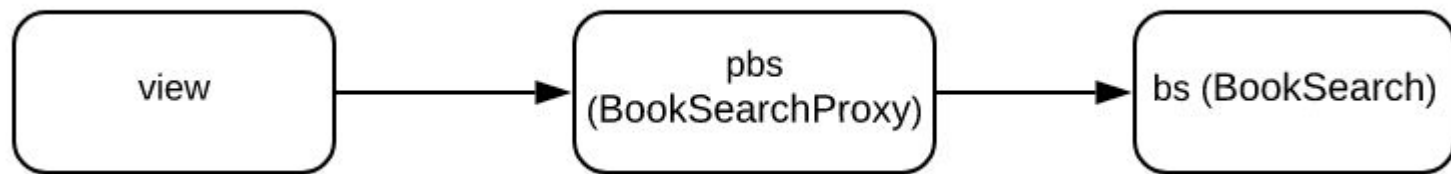
```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```



```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```



```
void main() {  
    BookSearch bs = new BookSearch();  
    BookSearchProxy pbs;  
    pbs = new BookSearchProxy(bs);  
    ...  
    View view = new View(pbs);  
    ...  
}
```





Logo, um proxy:

- (1) ajuda a melhorar qual propriedade de projeto?
- (2) é um aplicação de qual princípio SOLID?

Integridade Conceitual; Information Hiding; Coesão; Acoplamento

Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation; Dependency Inversion

`</> C#`

```
public class ProdutoService
{
    public Produto Buscar(int id)
    {
        // verificar autorização

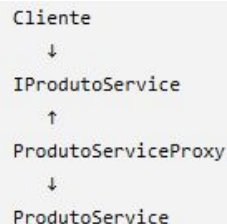
        // verificar cache

        // registrar log

        // consultar banco

        // atualizar cache

        return produto;
    }
}
```



```
public class ProdutoServiceProxy : IProdutoService
{
    private IProdutoService service;

    public ProdutoServiceProxy(IProdutoService service)
    {
        this.service = service;
    }

    public Produto Buscar(int id)
    {
        Console.WriteLine("Verificando cache...");

        return service.Buscar(id);
    }
}
```

4 - Adaptador

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code, likely from a programming language like JavaScript or Python, which adds a technical and digital feel to the overall aesthetic.

Contexto: Sistema para Controlar Projetores Multimídia

```
class ProjetorSamsung {  
    public void turnOn() { ... }  
    ...  
}  
  
class ProjetorLG {  
    public void enable(int timer) { ... }  
    ...  
}
```

Classes fornecidas pelos fabricantes dos projetores

Problema

- No sistema de controle de multimídias, eu gostaria de manipular uma única interface Projetor

```
interface Projetor {  
    void liga();  
}  
...  
class SistemaControleProjetores {  
    void init(Projetor projetor) {  
        projetor.liga(); // liga qualquer projetor  
    }  
}
```

Sendo mais técnico, eu gostaria de aproveitar e fazer uso de qual princípio SOLID?

Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation; Dependency Inversion

Problema (cont.)

```
class ProjetorSamsung {  
    public void turnOn() { ... }  
    ...  
}  
  
class ProjetorLG {  
    public void enable(int timer) { ... }  
    ...  
}
```

- São classes dos fabricantes dos projetores
- Eu não tenho acesso a elas. Para, por exemplo, fazer com que elas implementem a interface Projetor

Solução: **Padrão Adaptador**

Padrão (ou interface)
de entrada

Padrão (ou interface)
de saída



Classe Adaptadora

Solução: Implementar uma Classe Adaptadora

```
class AdaptadorProjetoSamsung implements Projeto {  
  
    private ProjetoSamsung projetor;  
  
    AdaptadorProjetoSamsung (ProjetoSamung projetor) {  
        this.projetor = projetor;  
    }  
  
    public void liga() {  
        projetor.turnOn();  
    }  
  
}
```

Solução: Implementar uma Classe Adaptadora

```
class AdaptadorProjetoSamsung implements Projeto {  
    private ProjetoSamsung projetor;  
  
    AdaptadorProjetoSamsung (ProjetoSamung projetor) {  
        this.projetor = projetor;  
    }  
  
    public void liga() {  
        projetor.turnOn();  
    }  
}
```

Projektor

ProjektorSamsung

AdaptadorProjektorSamsung



5 - Fachada

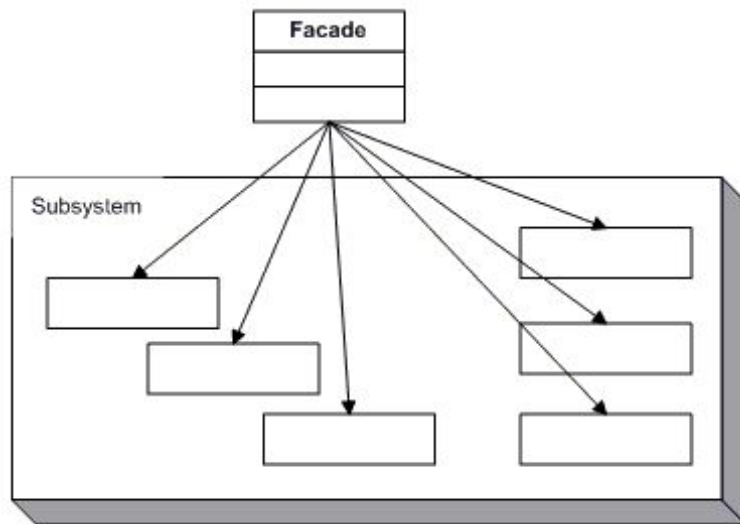
The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming theme. The overall aesthetic is modern and digital.

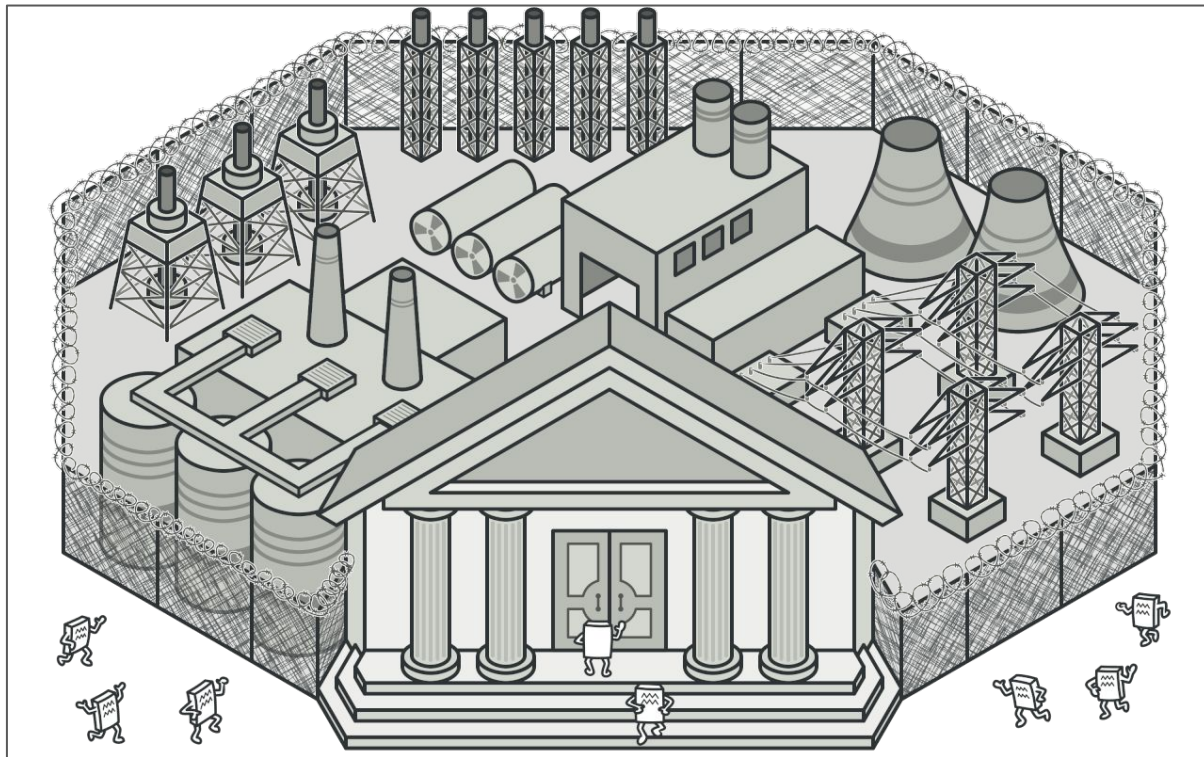
Contexto & Problema

- Contexto: módulo M usado por diversos outros módulos
- Problema:
 - Interface de M é complexa
 - Clientes reclamam que é difícil usar a interface de M

Solução

- Criar uma interface mais simples para M, chamada de **Fachada**.





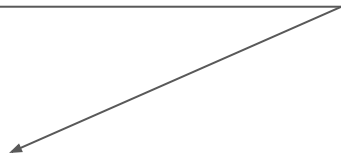
Exemplo: Interpretador

```
Scanner s = new Scanner("prog1.x");  
Parser p = new Parser(s);  
AST ast = p.parse();  
CodeGenerator code = new CodeGenerator(ast);  
code.eval();
```

```
Scanner s = new Scanner("prog1.x");  
Parser p = new Parser(s);  
AST ast = p.parse();  
CodeGenerator code = new CodeGenerator(ast);  
code.eval();
```

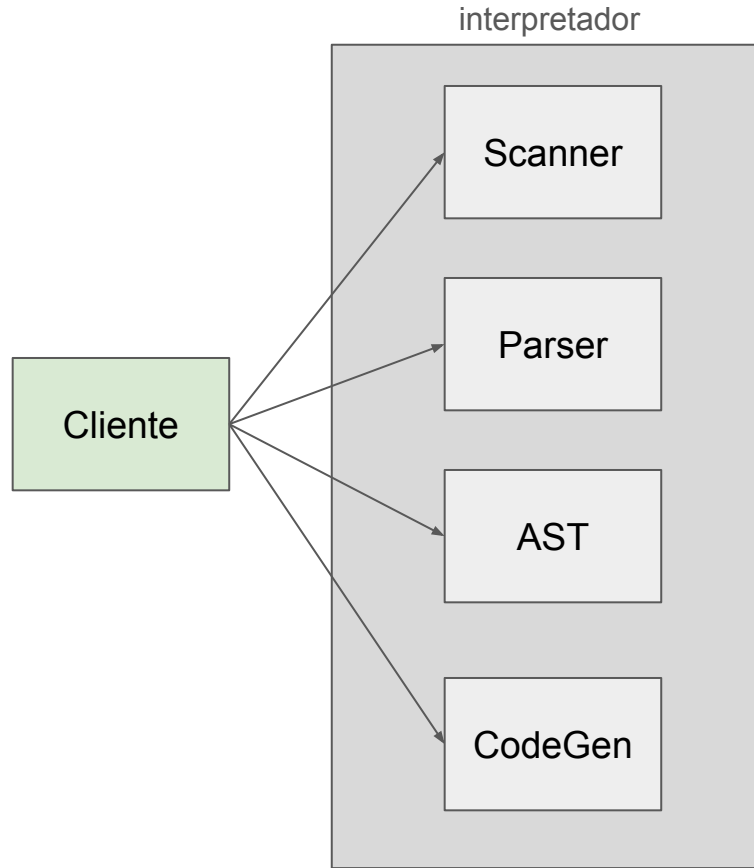


```
new InterpretadorX("prog1.x").eval();
```

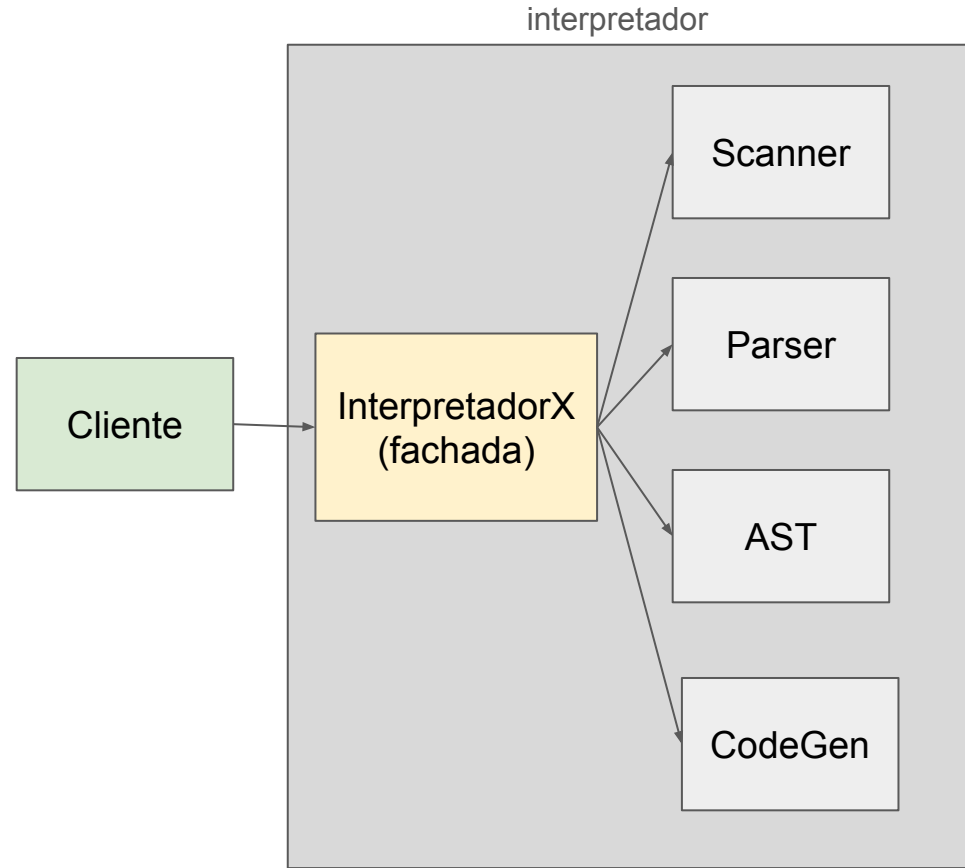


Interface mais simples,
que a de cima

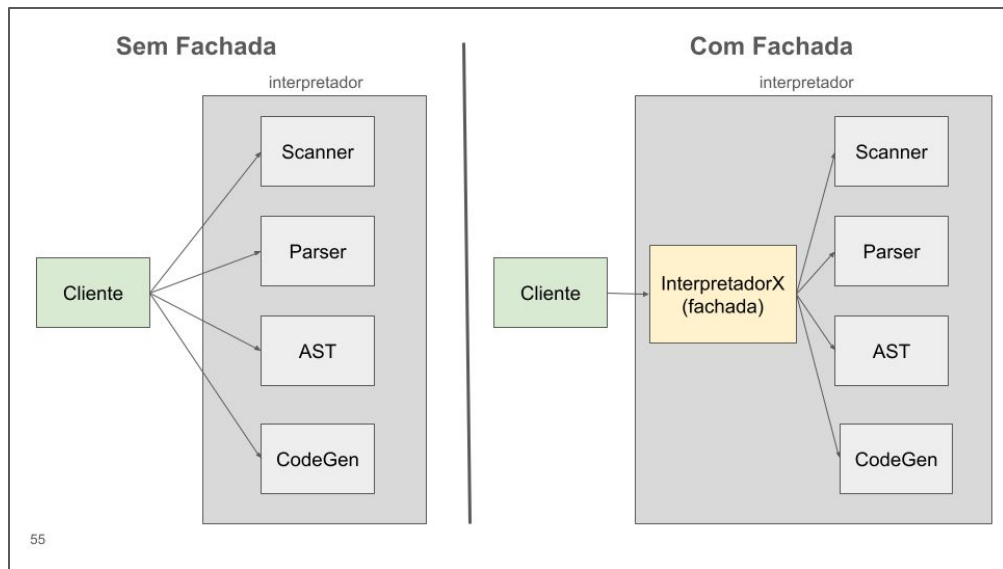
Sem Fachada



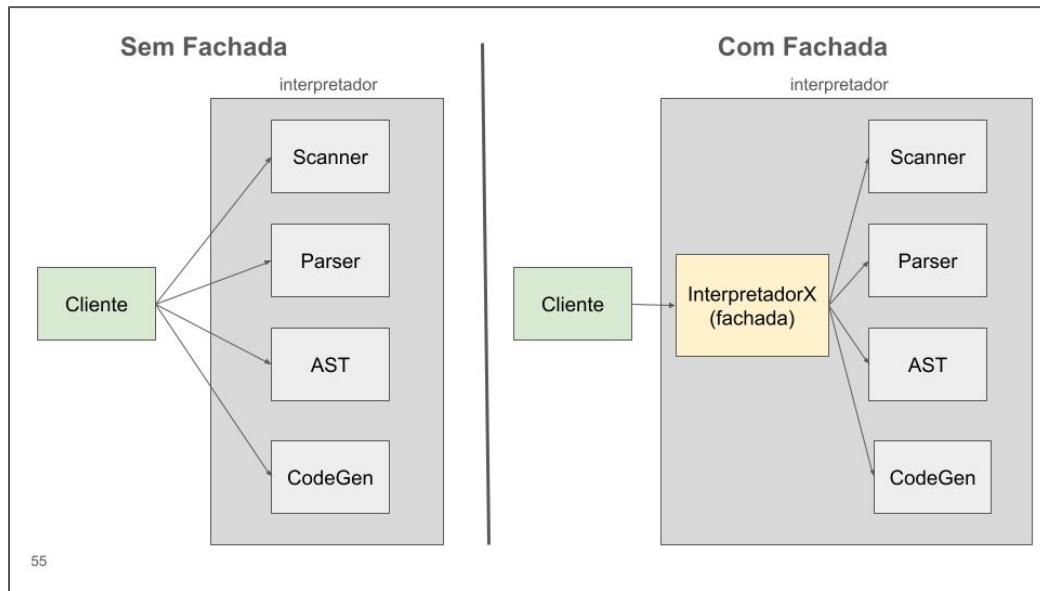
Com Fachada



Na figura anterior, considerando o cliente, qual propriedade do seu projeto estamos melhorando?



Por outro lado, se **não** tomarmos cuidado, qual princípio de projeto pode ser violado quando se usa uma fachada?



Single Responsibility; Open-Closed; Liskov Substitution; Interface Segregation; Dependency Inversion

Exercícios

1. Singleton é um dos padrões de projeto mais criticados e polêmicos. Erich Gamma, um dos autores do livro GoF, já chegou inclusive a recomendar, em uma [entrevista](#), que ele deveria ser eliminado do catálogo:

> Erich: When discussing which patterns to drop, we found that we still love them all. (Not really—I'm in favor of dropping Singleton...)

Descreva porque esse padrão pode causar problemas de design, caso não seja usado de forma adequada.

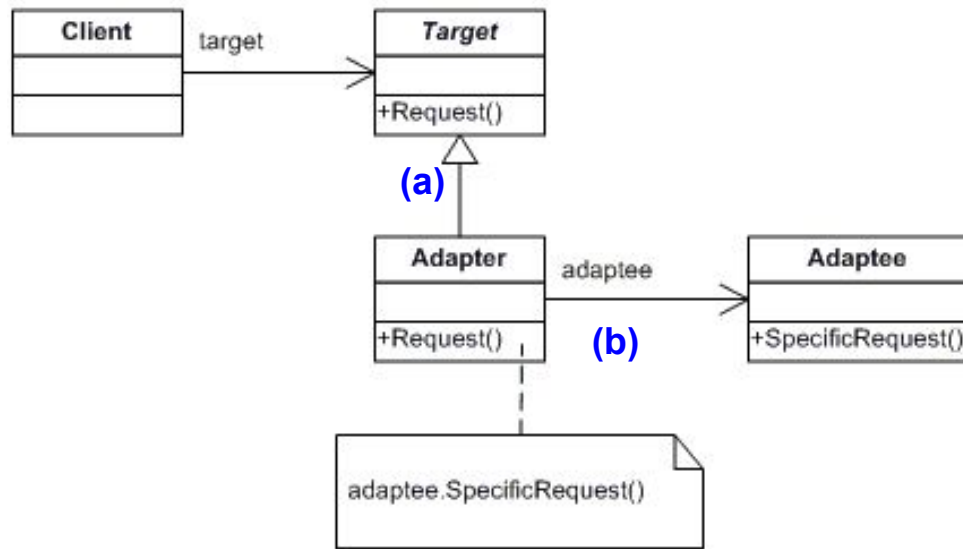
2. Por que a implementação de Singleton mostrada nos slides não funciona com sistemas concorrentes (multi-thread)? Descreva um erro que pode ocorrer quando ela é usada com esse tipo de sistema.

3. Além de otimização de desempenho via um cache, tal como visto nos slides, descreva outros três requisitos não-funcionais que podem ser implementados em um Proxy.

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

4. O seguinte diagrama de classes explica o padrão Adaptador.

- Em UML, qual o tipo da relação (a)? E qual o tipo da relação (b)?
- Mapeie Target, Adapter e Adaptee para as interfaces e classes do exemplo de projetores visto em aula.



6 - Decorador

The background features a light blue gradient with several wavy, horizontal bands of varying shades of blue and purple. These bands are composed of blurred, overlapping lines of code in various programming languages, including JavaScript, Python, and Java. The code snippets are partially legible, showing functions, loops, and class definitions. The overall effect is a sense of dynamic movement and digital complexity.

Contexto: Sistema que usa canais de comunicação

(já foi usado para explicar o padrão Fábrica)

```
interface Channel {  
    void send(String msg);  
    String receive();  
}  
  
class TCPChannel implements Channel {  
    ...  
}  
  
class UDPChannel implements Channel {  
    ...  
}
```

Problema: Precisamos adicionar funcionalidades extras em canais

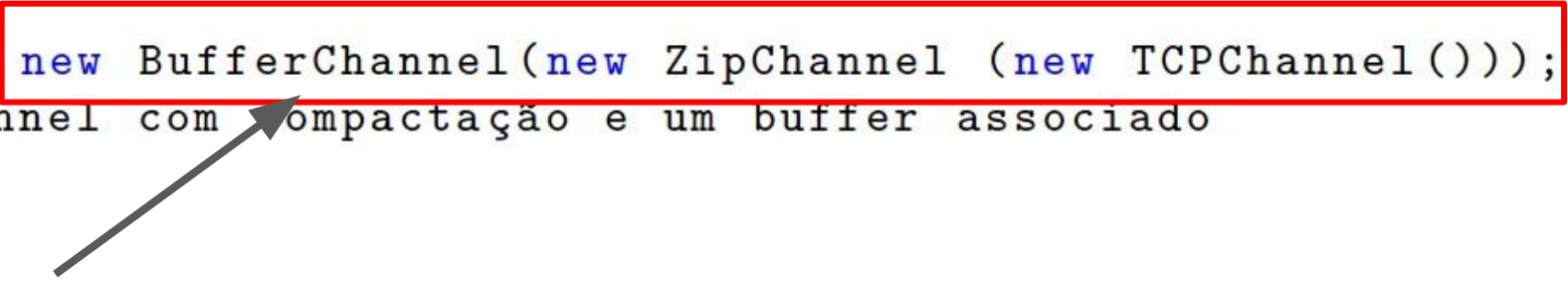
- Os canais default (TCP, UDP) não são suficientes
- Precisamos também de canais com:
 - Compactação e descompactação
 - Buffers (ou caches)
 - Logging
 - etc

Solução: **Padrão Decorador**

- Resolve esse problema -- adicionar funcionalidades em uma classe -- por meio de composição e sem gerar um número excessivo de classes

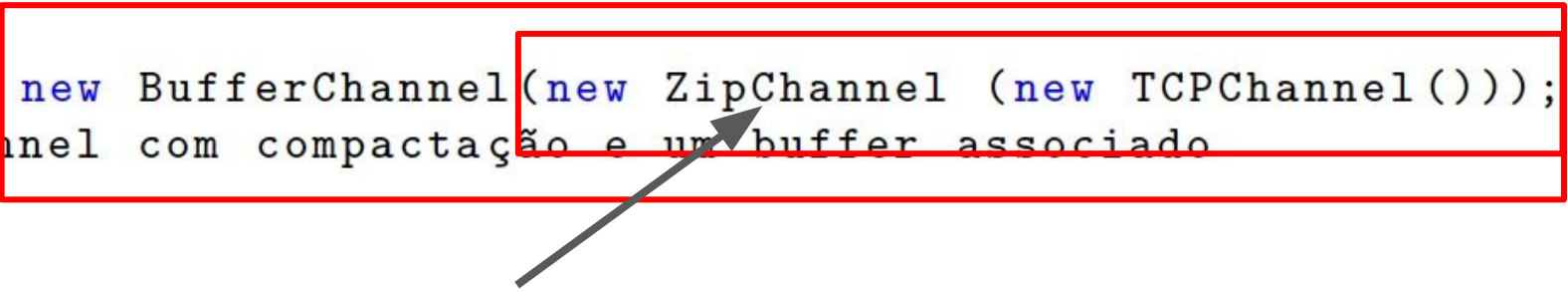
Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



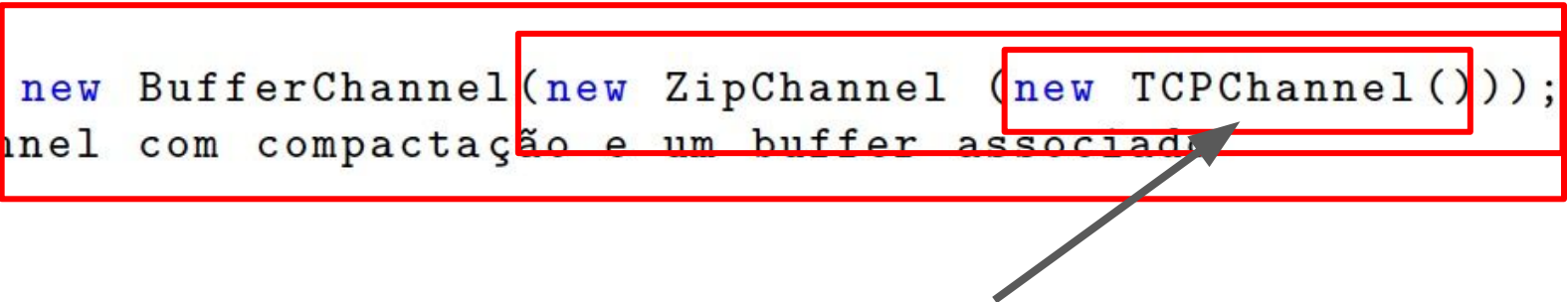
Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



Mais um exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



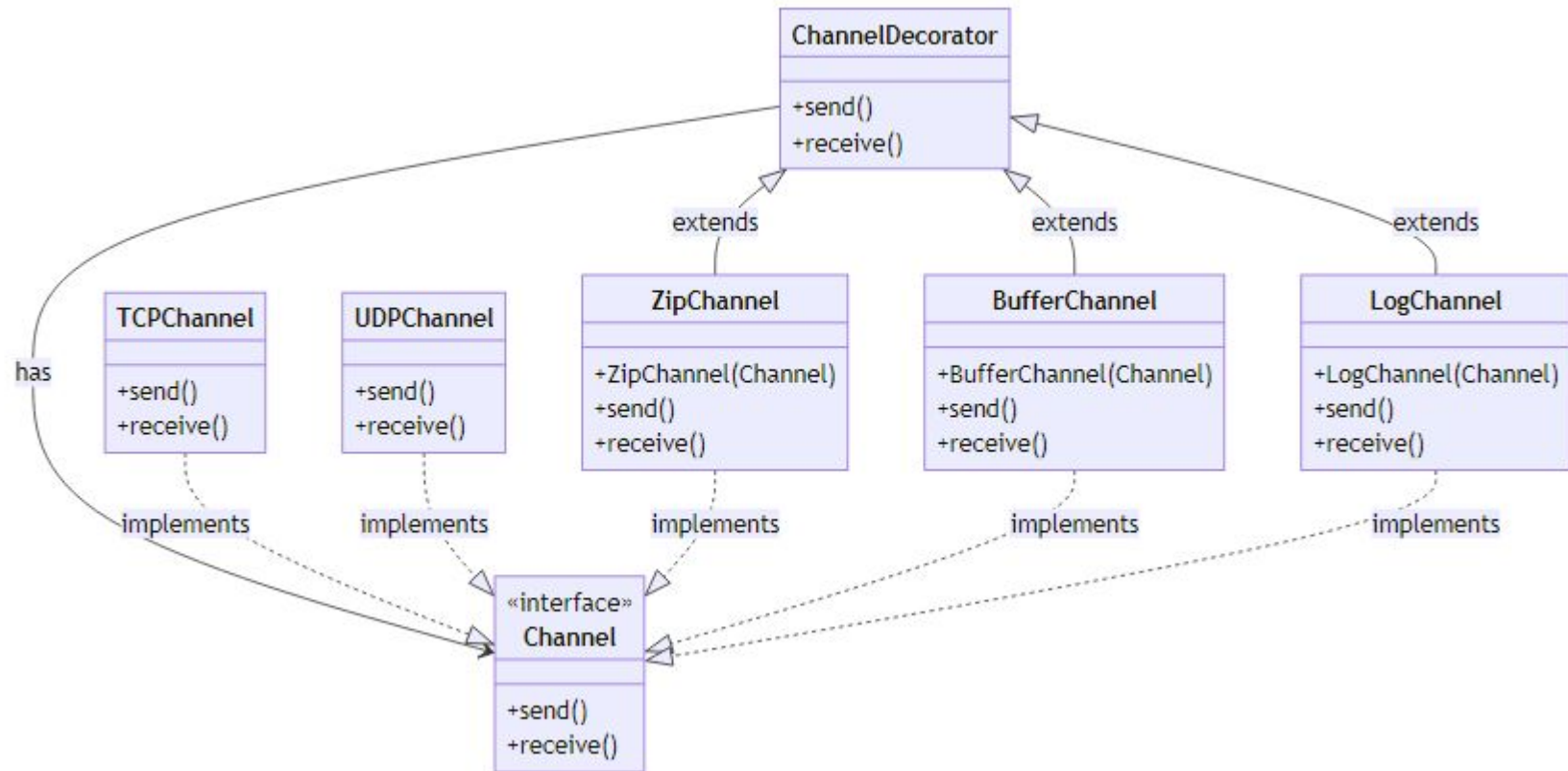
Comparação

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



Dentro de uma caixa, tem outra caixa, que tem outra caixa ... até chegar no presente. Isto é, até chegar em TCPChannel ou UDPChannel

Implementação do Padrão Decorador



ChannelDecorator

```
class ChannelDecorator implements Channel {  
  
    protected Channel channel;  
  
    public ChannelDecorator(Channel channel) {  
        this.channel = channel;  
    }  
  
    public void send(String msg) {  
        channel.send(msg);  
    }  
  
    public String receive() {  
        return channel.receive();  
    }  
  
}
```

Todos os decoradores são subclasses dessa classe única

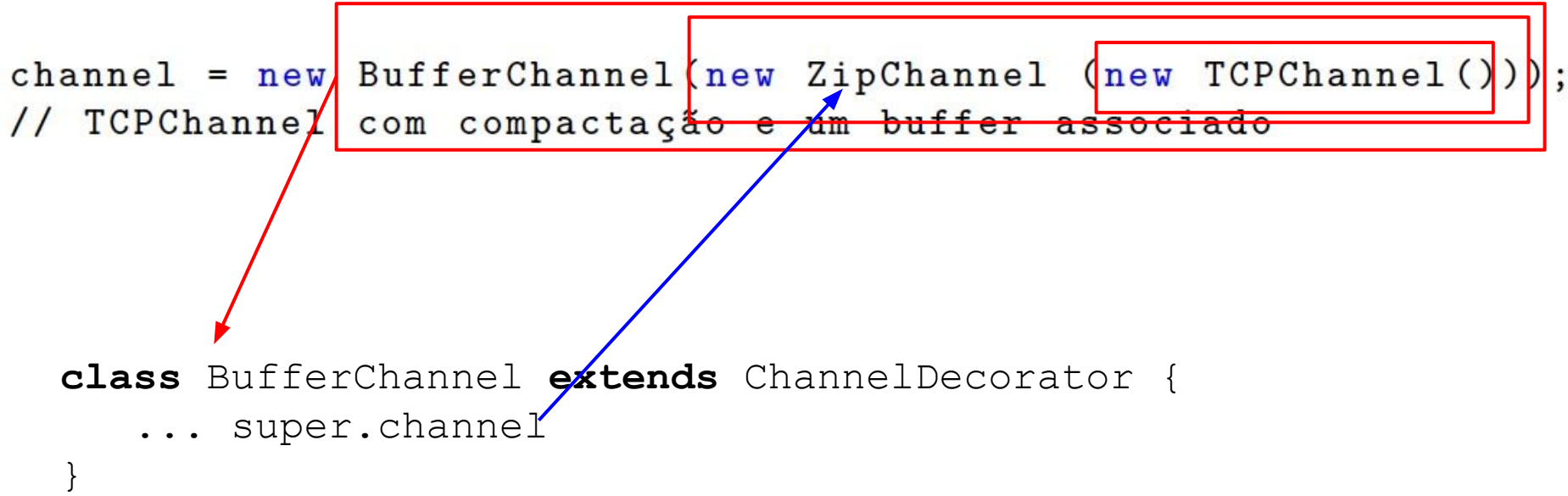
Implementação: ZipChannel

ZipChannel

```
class ZipChannel extends ChannelDecorator {  
  
    public ZipChannel(Channel c) {  
        super(c);  
    }  
  
    public void send(String msg) {  
        "compacta mensagem msg"  
        super.channel.send(msg);  
    }  
  
    public String receive() {  
        String msg = super.channel.receive();  
        "descompacta mensagem msg"  
        return msg;  
    }  
  
}
```

Exemplo

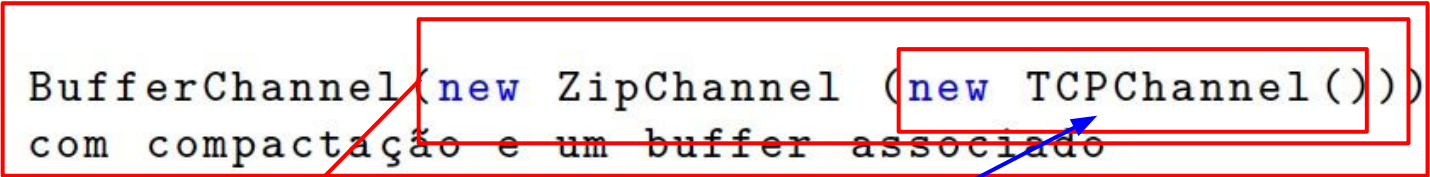
```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



```
class BufferChannel extends ChannelDecorator {  
    ... super.channel  
}
```

Exemplo

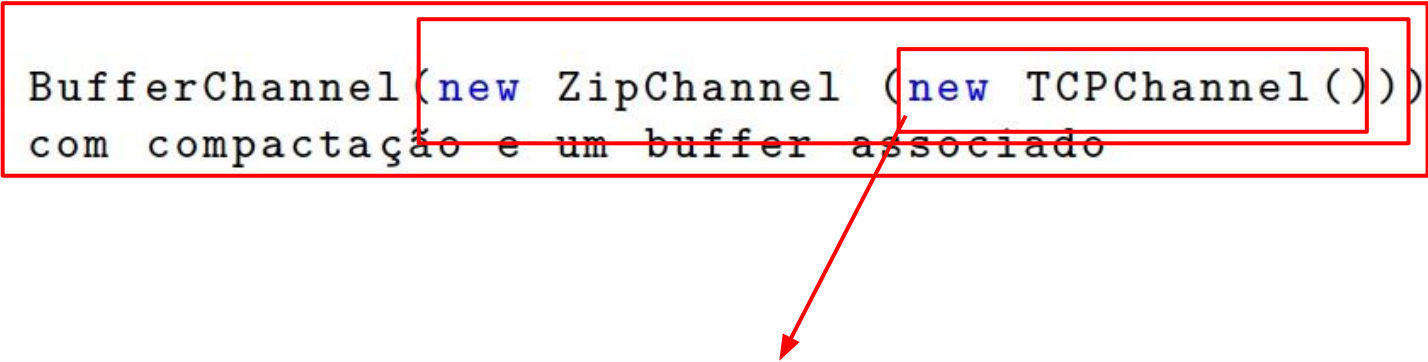
```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```



```
class ZipChannel extends ChannelDecorator {  
    ... super.channel  
}
```

Exemplo

```
channel = new BufferChannel(new ZipChannel (new TCPChannel()));  
// TCPChannel com compactação e um buffer associado
```

The code is enclosed in a large red box. Inside this box, the expression `new ZipChannel (new TCPChannel())` is further enclosed in a smaller red box. Below the main box, there is a red arrow pointing from the `new TCPChannel()` part of the code to the `class TCPChannel implements Channel` definition below.

```
class TCPChannel implements Channel {  
    // canal "final"  
}
```

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

